Complete finite programs for the three enumeration applications

Human Compression project

The eight programs below are retained in full. Five are required by the finite classification proofs: the degree-six census and the four Hall–Janko programs. The formula and two Suzuki programs are additional diagnostics, conservatively included in the complete program boundary. Their finite tests do not prove the formulas for arbitrary degrees or field exponents. The two full execution drivers are separately inventoried; the selected modes run only these three applications. No source classification table is used as a filter on the census.

1 The complete degree-six classification

```
# Full finite exceptional census. No published classification filters the search.
# Run from any directory; this program writes only standard output.
SizeScreen([4096,4096]);;
Need:=function(test,message) if not test then Error(message); fi; end;;
PairAction:=function(pair,phi) return Set(pair,x->Image(phi,x)); end;;
MiddleAction:=function(x,phi) return Image(phi,x); end;;
StringInts:=function(xs) return
 ↪  Concatenation("[",JoinStringsWithSeparator(List(xs,String),","),"]"); end;;
EncodePerm:=function(p) return StringInts(List([1..6],i->i^p)); end;;
EncodeTriple:=function(t) return
 ↪  Concatenation("[",JoinStringsWithSeparator(List(t,EncodePerm),","),"]"); end;;
AllIP:=function(G,triple)
  local subs,groups,i,j,ok,meets;
  subs:=Combinations([1..3]);
  groups:=List(subs,I->Subgroup(G,List(I,i->triple[i])));
  ok:=true;meets:=0;
  for i in [1..8] do for j in [1..8] do
    meets:=meets+1;
    if Intersection(groups[i],groups[j])<>
       groups[Position(subs,Intersection(subs[i],subs[j]))] then ok:=false;fi;
  od;od;
  return rec(ok:=ok,meets:=meets);
end;;
sourcePairs:=[[(1,2),(3,4)],[(1,2),(3,4)(5,6)],
 [(1,2),(1,2)(3,4)],[(1,2),(1,2)(3,4)(5,6)],
 [(1,2)(3,4),(1,3)(2,4)]];;
explicit:=[[(5,6),(2,3)(4,5),(1,2)(3,4)],
 [(5,6),(2,3)(4,5),(1,2)(3,4)(5,6)]];;
orbs:=[];;
for G in [SymmetricGroup(6),AlternatingGroup(6)] do
  inv:=Filtered(Elements(G),x->Order(x)=2);;
  Need(Size(Subgroup(G,inv))=Size(G),"Involutions do not generate ambient group");
  aut:=AutomorphismGroup(G);;
  pairs:=[];;
  for i in [1..Length(inv)] do for j in [i+1..Length(inv)] do
    if inv[i]*inv[j]=inv[j]*inv[i] then Add(pairs,Set([inv[i],inv[j]]));fi;
  od;od;
  orbs:=OrbitsDomain(aut,pairs,PairAction);;
  conjugacy:=OrbitsDomain(SymmetricGroup(6),pairs,OnSets);;
  if Size(G)=720 then reps:=List(sourcePairs,Set);else reps:=[Set(sourcePairs[5])];fi;
  repIndices:=List(reps,p->PositionProperty(orbs,o->p in o));;
  Need(not fail in repIndices,"Primary pair missing");
  Need(Set(repIndices)=[1..Length(orbs)] and Length(Set(repIndices))=Length(reps),
       "Primary representatives are not a complete irredundant orbit list");
  found:=[];;middleCount:=0;;generated:=0;;meetCount:=0;;extendable:=[];;
  for ri in [1..Length(reps)] do
    pair:=reps[ri];;a:=pair[1];;c:=pair[2];;
    stabilizer:=Stabilizer(aut,pair,PairAction);;
```

```
    middles:=OrbitsDomain(stabilizer,inv,MiddleAction);;
    localGenerated:=0;;localIP:=0;;
    for mo in middles do
      b:=mo[1];;middleCount:=middleCount+1;;
      if Size(Subgroup(G,[a,b,c]))=Size(G) then
        Need(Length(Set([a,b,c]))=3,"Generating triple has repeated marker");
        generated:=generated+1;;localGenerated:=localGenerated+1;;
        ip:=AllIP(G,[a,b,c]);;meetCount:=meetCount+ip.meets;;
        if ip.ok then
          localIP:=localIP+1;;typ:=SortedList([Order(a*b),Order(b*c)]);;
          Add(found,rec(triple:=[a,b,c],type:=typ,pair_index:=ri));
          Print("SURVIVOR|",Size(G),"|",ri,"|",StringInts(typ),"|",EncodeTriple([a,b,c]),"\
           ↪  n");
        fi;
      fi;
    od;
    if localGenerated>0 then Add(extendable,ri);fi;
    Print("FIBER|",Size(G),"|",ri,"|",Length(orbs[repIndices[ri]]),"|",Size(stabilizer),"|"
     ↪  ,Length(middles),"|",localGenerated,"|",localIP,"\n");
  od;
  explicitMeets:=0;;
  if Size(G)=720 then
    Need(Length(inv)=75 and Length(pairs)=495 and Length(conjugacy)=9 and
         Length(orbs)=5 and Size(aut)=1440,"Symmetric endpoint census mismatch");
    Need(middleCount=80 and generated=2 and Length(found)=2 and
         SortedList(List(found,r->r.type))=[[5,6],[6,6]] and extendable=[2,4],
         "Symmetric polyhedron census mismatch");
    for triple in explicit do
      Need(Size(Subgroup(G,triple))=Size(G),"Explicit triple does not generate");
      Need(triple[1]*triple[3]=triple[3]*triple[1],"Explicit endpoints fail string
       ↪  relation");
      ip:=AllIP(G,triple);;Need(ip.ok,"Explicit triple IP failure");
      explicitMeets:=explicitMeets+ip.meets;;
      typ:=SortedList([Order(triple[1]*triple[2]),Order(triple[2]*triple[3])]);;
      Need(typ in List(found,r->r.type),"Explicit type not in complete census");
    od;
  else
    Need(Length(inv)=45 and Length(pairs)=90 and Length(conjugacy)=2 and
         Length(orbs)=1 and Size(aut)=1440,"Alternating endpoint census mismatch");
    Need(middleCount=8 and generated=0 and Length(found)=0,
         "Alternating nonexistence census mismatch");
  fi;
  Print("SUMMARY|",Size(G),"|",Size(aut),"|",Length(inv),"|",Length(pairs),"|",Length(conju
   ↪  gacy),"|",Length(orbs),"|",middleCount,"|",generated,"|",Length(found),"|",meetCount,
   ↪  "|",explicitMeets,"\n");
od;
Print("VERSION|",GAPInfo.Version,"\nPASS\n");
QUIT;
```

2 All-degree formula diagnostics

```
#!/usr/bin/env python3
"""Optional exact controls; the manuscript proves the uniform statements."""
import argparse
import hashlib
import json
from pathlib import Path

def need(ok, message):
    if not ok:
        raise RuntimeError(message)

def geometric(degrees, limit):
    a = [1] + [0] * limit
    for d in degrees:
        for n in range(d, limit + 1):
            a[n] += a[n-d]
    return a

def series(limit):
    s = geometric([1, 1, 1, 2, 2], limit)
```

```
    sb = geometric([1, 1], limit)
    a = geometric([1, 2, 2, 2, 4], limit)
    ab = geometric([1, 2], limit)
    return ([s[n]-sb[n]-(sb[n-1] if n else 0) for n in range(limit+1)],
            [a[n]+(a[n-3] if n >= 3 else 0)-ab[n]
             -(ab[n-2] if n >= 2 else 0) for n in range(limit+1)])

def lam(k):
    return (k+2)**2//4

def psi(k, n):
    if k <= n//2:
        return 0
    h = 2*k-n
    return (h*h+2*h)//4 if n % 2 == 0 else (h-1)**2//4+h

def formulas(n):
    twice_s = -3*n+sum(lam(k)*(n-k+2)-psi(k, n) for k in range(1, n+1))
    mu = -2*(n//2)
    d = 0
    for k in range(2, n+1, 2):
        gamma, delta = (k*k+6*k+8)//8, (k*k+2*k)//8
        mu += gamma*((n-k+2)//2)+delta*((n-k+1)//2)
        d += lam(k)-psi(k, n)-1
    need(twice_s % 2 == (mu+d) % 2 == 0, 'Nonintegral formula')
    return twice_s//2, (mu+d)//2

def direct(n):
    s = a = 0
    for t in range(n//2+1):
        for x in range(n+1):
            for y in range(x, n-x-2*t+1):
                for z in range(n-x-y-2*t+1):
                    if not (t > 0 or sum(u > 0 for u in (x, y, z)) >= 2):
                        continue
                    s += 1
                    a += x % 2 == y % 2 == z % 2
    return s, a

def run():
    s, a = series(500)
    for n in range(501):
        need(formulas(n) == (s[n], a[n]), 'Floor/series disagreement at '+str(n))
    for n in range(31):
        need(direct(n) == (s[n], a[n]), 'Direct count disagreement at '+str(n))
    coordinates = 0
    for degree in range(4, 42):
        n = degree//2
        for k in range(1, n+1):
            for l in range(k+1):
                for t in range(l//2+1):
                    for x in range(n-k+1):
                        z, y, f = l-2*t, k-l, degree-2*(x+k)
                        need(min(z,y,f) >= 0 and f+2*(x+y+z)+4*t == degree,
                             'Coordinate inverse failed')
                        need(k == y+z+2*t and l == z+2*t, 'Coordinate forward failed')
                        faithful = t > 0 or sum(u > 0 for u in (x,y,z)) >= 2
                        removed = (x == l == t == 0) or (x == t == 0 and l == k)
                        need(faithful != removed, 'Faithfulness subtraction failed')
                        need((x == y) == (k == x+z+2*t), 'Swap-fixed condition failed')
                        need((k % 2 == 0 and (l+x) % 2 == 0) ==
                             (x % 2 == y % 2 == z % 2), 'Sign condition failed')
                        coordinates += 1
    table = [([1,2,3],0,0),([4,5],3,1),([6],5,1),([7],9,2),([8,9],21,7),
             ([10,11],39,10),([12,13],67,21),([14,15],105,28),([16,17],158,48),
            ([18,19],226,61),([20],315,93),([30],1169,315),([40],3105,855),([50],6774,1795)]
    for degrees, sn, an in table:
        for degree in degrees:
            value = (5,1) if degree == 6 else (s[degree//2],a[degree//2])
            need(value == (sn,an), 'Printed table mismatch')
    return
     ↪  dict(status='PASS',floor_and_series_half_degrees=[0,500],direct_half_degrees=[0,30],
                reversible_coordinates_checked=coordinates,printed_rows=len(table),
```

```
                degree_six_values='Supplied by separate complete GAP census, not inferred
                 ↪  from coefficients',
                uniform_proof='K(e) and CPI3-CPI9; bounded controls are supplementary',
                program_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest())

def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--output', type=Path, required=True)
    args = p.parse_args()
    need(not args.output.exists(), 'Refusing existing output')
    report = run()
    with args.output.open('x') as f:
        json.dump(report, f, indent=2)
        f.write('\n')
    print(json.dumps(report))

if __name__ == '__main__':
    main()
```

3 Suzuki field-coordinate diagnostics

```
#!/usr/bin/env python3
"""Exact arithmetic and small-field orbit checks for marked Suzuki triples.

This does not prove the retained Suzuki-group structure or extrapolate group
generation from samples. The all-parameter equivalences are in the written note.
Only Python's standard library is required. Outputs never contain source text.
"""
import argparse
from collections import Counter
from datetime import datetime, timezone
from fractions import Fraction
import hashlib
import json
import math
from pathlib import Path


def divisors(n):
    return [d for d in range(1, n + 1) if n % d == 0]


def mobius(n):
    result = 1
    p = 2
    while p * p <= n:
        if n % p == 0:
            n //= p
            result = -result
            if n % p == 0:
                return 0
            while n % p == 0:
                n //= p
        p += 1
    return -result if n > 1 else result


def lam(n):
    return Fraction(sum(mobius(n // d) * (1 << d) for d in divisors(n)), n)


def psi(n, total):
    return sum((Fraction(sum(mobius(m // d) * ((1 << (n*d)) - 1)
                             for d in divisors(m)), m)
                for m in divisors(total // n)), Fraction(0))


def all_orbits(n):
    return sum((lam(d) * psi(d, n) for d in divisors(n) if d > 1), Fraction(0))


def compact(n):
```

```
    return Fraction(sum(mobius(n // d) * ((1 << d) - 2) * ((1 << d) - 1)
                        for d in divisors(n)), 2*n)


class BinaryField:
    def __init__(self, degree, modulus):
        self.degree = degree
        self.modulus = modulus
        self.q = 1 << degree
        assert modulus.bit_length() == degree + 1
        # Finite verification that every nonzero residue is a unit suffices
        # here to certify these three small polynomial quotient fields.
        assert all(self.pow(a, self.q-1) == 1 for a in range(1, self.q))
        self.square = [self.mul(a, a) for a in range(self.q)]
        self.inverse = [0] + [self.pow(a, self.q-2) for a in range(1, self.q)]

    def mul(self, a, b):
        result = 0
        while b:
            if b & 1:
                result ^= a
            a <<= 1
            b >>= 1
            if a & self.q:
                a ^= self.modulus
        return result

    def pow(self, a, exponent):
        result = 1
        while exponent:
            if exponent & 1:
                result = self.mul(result, a)
            a = self.mul(a, a)
            exponent >>= 1
        return result

    def frobenius_orbit(self, point):
        result = []
        current = point
        while current not in result:
            result.append(current)
            current = tuple(self.square[a] for a in current)
        assert current == point and self.degree % len(result) == 0
        return result

    def dual(self, point):
        b, c = point
        return (self.inverse[b], self.mul(b, c))


def field_check(n, modulus):
    field = BinaryField(n, modulus)
    seen = set()
    representatives = {}
    degree_counts = Counter()
    for b in range(2, field.q):
        for c in range(1, field.q):
            point = (b, c)
            assert field.dual(field.dual(point)) == point
            squared = (field.square[b], field.square[c])
            assert field.dual(squared) == tuple(field.square[a] for a in field.dual(point))
            if point in seen:
                continue
            orbit = field.frobenius_orbit(point)
            seen.update(orbit)
            key = min(orbit)
            dual_key = min(field.frobenius_orbit(field.dual(point)))
            assert key != dual_key
            representatives[key] = dual_key
            degree_counts[len(orbit)] += 1
    assert len(seen) == (field.q - 2) * (field.q - 1)
    assert len(representatives) == all_orbits(n)
    assert all(representatives[value] == key for key, value in representatives.items())
    for d, count in degree_counts.items():
```

```
        assert count == 2 * compact(d)

    pair_seen = set()
    pair_orbits = 0
    for b in range(2, field.q):
        if b in pair_seen:
            continue
        orbit = [a[0] for a in field.frobenius_orbit((b,))]
        pair_seen.update(orbit)
        assert field.inverse[b] not in orbit
        pair_orbits += 1
    assert pair_orbits == sum(lam(d) for d in divisors(n) if d > 1)
    return dict(degree=n, q=field.q, polynomial_bits=modulus,
                marked_pairs=len(seen), frobenius_orbits=len(representatives),
                orbits_by_exact_joint_degree=dict(sorted(degree_counts.items())),
                full_group_duality_classes=degree_counts[n] // 2,
                unordered_commuting_pair_classes=pair_orbits // 2,
                checked_duality_involutivity_and_frobenius_commutation=True)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output', type=Path)
    args = parser.parse_args()
    arithmetic = []
    for n in range(3, 100, 2):
        nested = sum((mobius(n // d) * all_orbits(d) for d in divisors(n)), Fraction(0)) / 2
        simple = compact(n)
        assert nested == simple and simple.denominator == 1 and simple > 0
        arithmetic.append(dict(degree=n, compact_count=int(simple),
         ↪  nested_count=int(nested)))
    fields = [field_check(3, 0b1011), field_check(5, 0b100101),
              field_check(9, (1 << 9) | (1 << 4) | 1)]
    report = dict(status='PASS', executed_utc=datetime.now(timezone.utc).isoformat(),
                  source_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
                  arithmetic='Exact integers and rational numbers; no floating-point
                   ↪  comparisons',
                  formula_comparisons=arithmetic, finite_field_orbit_checks=fields,
                  scope='Finite formula/orbit diagnostics only. Uniform formula equivalence,
                   ↪  Suzuki normalization, generation and subgroup intersection are written
                   ↪  proofs with explicit retained group-structure inputs.')
    if args.output:
        args.output.write_text(json.dumps(report, indent=2) + '\n')
    print(json.dumps(dict(status='PASS', odd_degrees_compared=len(arithmetic),
     ↪  fields=fields), indent=2))


if __name__ == '__main__':
    main()
```

4 Suzuki matrix diagnostics

```
# Exact q=8 matrix/group diagnostics; no source-paper counts are inputs.
# Run with gap -q -T phase10_suzuki_marked_groups_check.g.
F:=GF(8);; z0:=Zero(F);; e:=One(F);;
Zroot:=function(b)
  return [[e,z0,z0,z0],[z0,e,z0,z0],[b,z0,e,z0],[b^4,b,z0,e]];
end;;
w:=[[z0,z0,z0,e],[z0,z0,e,z0],[z0,e,z0,z0],[e,z0,z0,z0]];;
G:=SuzukiGroup(IsMatrixGroup,8);;
Assert(0,Size(G)=29120);
Assert(0,w in G and ForAll(Elements(F),b->Zroot(b) in G));
iso:=IsomorphismPermGroup(G);;
P:=Image(iso);;
indices:=Combinations([1..3]);;
tripleCount:=0;; intersectionCount:=0;; dualMatrixCount:=0;;
types:=[];;
rho:=[];; h:=IdentityMat(4,F);;
for b in Difference(Elements(F),[z0,e]) do
  for c in Difference(Elements(F),[z0]) do
    triple:=[Zroot(e),w*Zroot(c)*w,Zroot(b)];;
    rho:=List(triple,m->Image(iso,m));;
```

```
    Assert(0,ForAll(rho,r->Order(r)=2));
    Assert(0,rho[1]*rho[3]=rho[3]*rho[1]);
    Assert(0,Group(rho)=P);
    subgroupList:=List(indices,I->Subgroup(P,rho{I}));;
    for I in [1..8] do
      for J in [1..8] do
        K:=Position(indices,Intersection(indices[I],indices[J]));;
        Assert(0,Intersection(subgroupList[I],subgroupList[J])=subgroupList[K]);
        intersectionCount:=intersectionCount+1;
      od;
    od;
    # h^-1 z(x) h = z(lambda^6*x); theta=4 for GF(8).
    lambda:=First(Difference(Elements(F),[z0]),a->a^6=b^-1);;
    h:=DiagonalMat([lambda^5,lambda,lambda^-1,lambda^-5]);;
    Assert(0,h in G);
    reversed:=Reversed(triple);;
    dual:=[Zroot(e),w*Zroot(b*c)*w,Zroot(b^-1)];;
    Assert(0,List(reversed,m->m^h)=dual);
    dualMatrixCount:=dualMatrixCount+1;
    Add(types,[Order(rho[1]*rho[2]),Order(rho[2]*rho[3])]);
    tripleCount:=tripleCount+1;
  od;
od;
Print("{\"status\":\"PASS\",\"q\":8,\"group_order\":",Size(P),
  ",\"normalized_triples\":",tripleCount,
  ",\"subgroup_intersections\":",intersectionCount,
  ",\"duality_matrix_checks\":",dualMatrixCount,
  ",\"adjacent_order_pairs\":",Set(types),"}\n");
QUIT;
```

5 The complete Hall–Janko search

```
# Complete arbitrary-B RWPRI/FT/RC/(2T)1/(IP)2 augmentation in subgroups of J2.
# Completeness input: all TomLib subgroup classes and maximal-inclusion classes.
# No original geometry representatives are read.
LoadPackage("tomlib");;
SetInfoLevel(InfoWarning,0);;
SizeScreen([100000,24]);;
tom:=TableOfMarks("J2");;G:=UnderlyingGroup(tom);;
reps:=List([1..Length(OrdersTom(tom))],i->RepresentativeTom(tom,i));;
maxorders:=[];;maxinclusions:=[];;local2T:=[];;
for K in reps do
 k:=Size(K);
 if not IsBound(maxorders[k]) then maxorders[k]:=[];fi;
 if k>1 then
  mm:=List(ConjugacyClassesMaximalSubgroups(K),Representative);
  UniteSet(maxorders[k],Set(List(mm,Size)));
  for H in mm do
   Add(maxinclusions,[K,H]);
   if Length(DoubleCosets(K,H,H))=2 then Add(local2T,[K,H]);fi;
  od;
 fi;
od;
Print("INPUT subgroup_classes=",Length(reps),"
 ↪  maximal_inclusion_classes=",Length(maxinclusions),"
 ↪  locally_2T_inclusions=",Length(local2T)," GAP=",GAPInfo.Version,"
 ↪  TomLib=",PackageInfo("tomlib")[1].Version,"\n");
IsMaxPair:=function(K,H)
 local ix,hom,p;
 if Size(K)=Size(H) or not Size(H) in maxorders[Size(K)] then return false;fi;
 if Size(K)=Size(G) then
  return ForAny(maxinclusions,p->Size(p[1])=Size(G) and Size(p[2])=Size(H) and
   ↪  RepresentativeAction(G,H,p[2],OnPoints)<>fail);
 fi;
 ix:=Index(K,H);
 if IsPrimeInt(ix) then return true;fi;
 if Size(K)>1000 then
  return ForAny(ConjugacyClassesMaximalSubgroups(K),p->Size(Representative(p))=Size(H) and
   ↪  RepresentativeAction(K,H,Representative(p),OnPoints)<>fail);
 fi;
 hom:=ActionHomomorphism(K,RightCosets(K,H),OnRight);
 return IsPrimitive(Image(hom),[1..ix]);
```

```
end;
OvergroupsFromInclusions:=function(H,inclusions)
 local ans,N,p,c;
 ans:=[];N:=Normalizer(G,H);
 for p in inclusions do
  if Size(p[2])=Size(H) then
   c:=RepresentativeAction(G,p[2],H,OnPoints);
   if c<>fail then UniteSet(ans,Orbit(N,p[1]^c,OnPoints));fi;
  fi;
 od;
 return ans;
end;
JoinPanels:=function(B,panels)
 local K,P;
 K:=B;
 for P in panels do K:=ClosureGroup(K,P);od;
 return K;
end;
FlagTransitive:=function(K,pars)
 local i,j,J,other,T,candidates,expected,rows;
 for i in [1..Length(pars)] do
  other:=Difference([1..Length(pars)],[i]);rows:=[];
  for j in other do
   T:=Intersection(pars[i],pars[j]);
   rows[j]:=Set(List(RightTransversal(pars[j],T),x->RightCoset(pars[i],x)));
  od;
  for J in Combinations(other) do
   if Length(J)<2 then continue;fi;
   candidates:=Intersection(List(J,j->rows[j]));
   T:=Intersection(List(J,j->pars[j]));
   expected:=Index(T,Intersection(T,pars[i]));
   if Length(candidates)<>expected then return false;fi;
  od;
 od;
 return true;
end;
# Locally two-transitive actions make a single common-neighbour count exact.
RankTwoAdmissible:=function(B,P,Q)
 local K,x,left,right;
 if Intersection(P,Q)<>B then return fail;fi;
 K:=ClosureGroup(P,Q);
 if not (IsMaxPair(K,P) or IsMaxPair(K,Q)) then return fail;fi;
 if Size(P)*Size(Q)<>Size(K)*Size(B) then
  x:=First(GeneratorsOfGroup(Q),x->not x in B);
  left:=Set(List(RightTransversal(P,B),h->RightCoset(Q,h)));
  right:=Set(List(RightTransversal(P,B),h->RightCoset(Q,h*x)));
  if Length(Intersection(left,right))<>1 then return fail;fi;
 fi;
 return K;
end;
ValidPanels:=function(B,panels)
 local n,ss,subs,S,T,U;
 n:=Length(panels);ss:=Combinations([1..n]);subs:=List(ss,S->JoinPanels(B,panels{S}));
 for S in [1..Length(ss)] do
  for T in [S+1..Length(ss)] do
   U:=Position(ss,Intersection(ss[S],ss[T]));
   if Intersection(subs[S],subs[T])<>subs[U] then return false;fi;
  od;
  if Length(ss[S])>=2 and not
   ↪  ForAny(ss[S],i->IsMaxPair(subs[S],subs[Position(ss,Difference(ss[S],[i]))])) then
   ↪  return false;fi;
 od;
 for S in ss do
  if Length(S)>=3 then
   U:=subs[Position(ss,S)];
   if not FlagTransitive(U,List(S,i->subs[Position(ss,Difference(S,[i]))])) then return
    ↪  false;fi;
  fi;
 od;
 return true;
end;
RunBorel:=function(bid)
 local B,N,panels,hom,A,orders,paircache,PairJoin,Equivalent,levels,rank,next,sigs,checked,
  ↪  ix,t,H,C,over,poss,orb,oo,x,u,K,sig,bucket,allJ2,checkpoint,maxrank;
```

```
B:=reps[bid];N:=Normalizer(G,B);panels:=OvergroupsFromInclusions(B,local2T);
Print("BOREL_BEGIN id=",bid," order=",Size(B)," normalizer=",Size(N),"
 ↪  panels=",Length(panels),"\n");
if Length(panels)=0 then
 PrintTo(Concatenation("research/leemans/phase5_j2_checkpoints/b",String(bid),"_done.g"),"
  ↪  borelResult:=rec(id:=",bid,",order:=",Size(B),",levels:=[],J2:=[]);\n");
 Print("BOREL_COMPLETE id=",bid," rank_counts=[] J2=[]\n");return;
fi;
hom:=ActionHomomorphism(N,panels,OnPoints);A:=Image(hom);orders:=List(panels,Size);paircac
 ↪  he:=[];
PairJoin:=function(i,j)
 local l,h,K;
 l:=Minimum(i,j);h:=Maximum(i,j);
 if not IsBound(paircache[l]) then paircache[l]:=[];fi;
 if not IsBound(paircache[l][h]) then
  ↪  paircache[l][h]:=RankTwoAdmissible(B,panels[l],panels[h]);fi;
 return paircache[l][h];
end;
Equivalent:=function(a,b)
 local p,ord,g,C,x,i;
 for p in SymmetricGroup(Length(a)) do
  ord:=Permuted(b,p);
  if orders{a}<>orders{ord} then continue;fi;
  if not ForAll(Combinations([1..Length(a)],2),ij->Size(PairJoin(a[ij[1]],a[ij[2]]))=Size(
   ↪  PairJoin(ord[ij[1]],ord[ij[2]]))) then continue;fi;
  g:=();C:=A;
  for i in [1..Length(a)] do
   x:=RepresentativeAction(C,a[i]^g,ord[i]);
   if x=fail then break;fi;
   g:=g*x;C:=Stabilizer(C,ord[i]);
  od;
  if x<>fail then return true;fi;
 od;
 return false;
end;
levels:=[List(Orbits(A,[1..Length(panels)]),o->[o[1]])];allJ2:=[];
Print("BOREL_LEVEL id=",bid," rank=1 classes=",Length(levels[1]),"
 ↪  J2=",Number(levels[1],t->panels[t[1]]=G),"\n");
for t in levels[1] do if panels[t[1]]=G then Add(allJ2,[panels[t[1]]]);fi;od;
maxrank:=Length(FactorsInt(Index(G,B)))+1;
for rank in [2..maxrank] do
 next:=[];sigs:=[];checked:=0;
 for ix in [1..Length(levels[rank-1])] do
  t:=levels[rank-1][ix];H:=JoinPanels(B,panels{t});C:=A;
  for x in t do C:=Stabilizer(C,x);od;
  over:=OvergroupsFromInclusions(H,maxinclusions);
  poss:=Filtered([1..Length(panels)],x->not IsSubgroup(H,panels[x]) and
   ↪  ForAny(over,K->IsSubgroup(K,panels[x])) and ForAll(t,y->PairJoin(x,y)<>fail));
  orb:=Orbits(C,poss);
  for oo in orb do
   x:=oo[1];u:=Concatenation(t,[x]);K:=First(over,K->IsSubgroup(K,panels[x]));checked:=che
    ↪  cked+1;
   sig:=[Size(K),SortedList(orders{u}),SortedList(List(Combinations(u,2),ij->Size(PairJoin
    ↪  (ij[1],ij[2]))))];
   bucket:=Filtered([1..Length(next)],i->sigs[i]=sig);
   if ForAny(bucket,i->Equivalent(next[i],u)) then continue;fi;
   if not ValidPanels(B,panels{u}) then continue;fi;
   Add(next,u);Add(sigs,sig);
   if K=G then Add(allJ2,panels{u});fi;
  od;
  if ix mod 10=0 or ix=Length(levels[rank-1]) then Print("BOREL_PROGRESS id=",bid,"
   ↪  rank=",rank," anchor=",ix,"/",Length(levels[rank-1])," checked=",checked,"
   ↪  accepted=",Length(next),"\n");fi;
 od;
 levels[rank]:=next;
 Print("BOREL_LEVEL id=",bid," rank=",rank," classes=",Length(next),"
  ↪  J2=",Number(next,t->JoinPanels(B,panels{t})=G),"\n");
 checkpoint:=Concatenation("research/leemans/phase5_j2_checkpoints/b",String(bid),"_r",Str
  ↪  ing(rank),".g");
 PrintTo(checkpoint,"borelCheckpoint:=rec(id:=",bid,",order:=",Size(B),",rank:=",rank,",pa
  ↪  nels:=",panels,",level:=",next,",J2:=",allJ2,");\n");
 if Length(next)=0 then break;fi;
od;
```

```
 if Length(levels[Length(levels)])<>0 then Error("chain bound reached without an empty
  ↪  level");fi;
 PrintTo(Concatenation("research/leemans/phase5_j2_checkpoints/b",String(bid),"_done.g"),"b
  ↪  orelResult:=rec(id:=",bid,",order:=",Size(B),",levels:=",List(levels,Length),",J2:=",a
  ↪  llJ2,");\n");
 Print("BOREL_COMPLETE id=",bid," rank_counts=",List(levels,Length),"
  ↪  J2_ranks=",List(allJ2,Length),"\n");
end;
if not IsBound(BOREL_IDS) then BOREL_IDS:=[1..Length(reps)];fi;
for bid in BOREL_IDS do RunBorel(bid);od;
Print("SEARCH_COMPLETE borel_ids=",BOREL_IDS," CPU_RUNTIME_MS=",Runtime(),"\n");
QUIT;
```

6 Full automorphism and type equivalence

```
# Fuse independently enumerated J2 geometries; no published geometry list read.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
G:=UnderlyingGroup(TableOfMarks("J2"));;A:=AutomorphismGroup(G);;
all:=[];;
for bid in [1..146] do
 Read(Concatenation("research/leemans/phase5_j2_checkpoints/b",String(bid),"_done.g"));
 if borelResult.id<>bid then Error("checkpoint id mismatch");fi;
 for t in borelResult.J2 do
  Add(all,rec(borel_id:=bid,borel_order:=borelResult.order,panels:=t,parabolics:=List([1..L
   ↪  ength(t)],i->Group(Concatenation(List(t{Difference([1..Length(t)],[i])},GeneratorsOfG
   ↪  roup))))));
 od;
od;
AutoSub:=function(H,a) return Image(a,H);end;
Signature:=function(p)
 return [SortedList(List(p,Size)),SortedList(List(Combinations([1..Length(p)],2),ij->Size(I
  ↪  ntersection(p[ij[1]],p[ij[2]])))),Size(Intersection(p))];
end;
EquivalentFamilies:=function(A,p,q)
 local perm,ord,cur,C,x,i;
 if Signature(p)<>Signature(q) then return false;fi;
 for perm in SymmetricGroup(Length(p)) do
  ord:=Permuted(q,perm);
  if List(p,Size)<>List(ord,Size) then continue;fi;
  cur:=ShallowCopy(p);C:=A;x:=fail;
  for i in [1..Length(p)] do
   x:=RepresentativeAction(C,cur[i],ord[i],AutoSub);
   if x=fail then break;fi;
   cur:=List(cur,H->Image(x,H));C:=Stabilizer(C,ord[i],AutoSub);
  od;
  if x<>fail then return true;fi;
 od;
 return false;
end;
allFinal:=[];;counts:=[];;
for rank in [2..14] do
 candidates:=Filtered(all,e->Length(e.panels)=rank);classes:=[];orbits:=[];
 for i in [1..Length(candidates)] do
  hit:=First([1..Length(classes)],j->EquivalentFamilies(A,candidates[classes[j]].parabolics
   ↪  ,candidates[i].parabolics));
  if hit=fail then Add(classes,i);Add(orbits,[i]);else Add(orbits[hit],i);fi;
 od;
 counts[rank]:=Length(classes);Append(allFinal,candidates{classes});
 Print("FUSION rank=",rank," inner_classes=",Length(candidates),"
  ↪  aut_type_classes=",Length(classes)," orbits=",orbits,"\n");
 Print("BOREL_CLASSES rank=",rank,"
  ↪  orders=",List(candidates{classes},e->e.borel_order),"\n");
od;
PrintTo("research/leemans/phase5_j2_representatives.g","# Independently enumerated J2
 ↪  geometry witnesses.\nG:=",G,";\nJ2Representatives:=",allFinal,";\n");
if counts{[2..4]}<>[8,19,3] or not ForAll([5..14],r->counts[r]=0) then Error("published
 ↪  principal census differs from independent result");fi;
Print("FULL_CENSUS_PASS counts=",counts{[2..14]}," aut_order=",Size(A),"\n");
QUIT;
```

7 Verification of all final geometries

```
# Independent parabolic recognition of all freshly enumerated final witnesses.
# The recognition code is reused; no phase-four geometry data are read.
Read("research/leemans/phase4_geometry_checks.g");;
Read("research/leemans/phase5_j2_representatives.g");;
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
if G<>UnderlyingGroup(TableOfMarks("J2")) then Error("witness group differs from named
 ↪  J2");fi;
for i in [1..Length(J2Representatives)] do
 entry:=J2Representatives[i];r:=VerifyGeometry(G,entry.parabolics);
 if not (r.firm and r.ft and r.rc and r.rwpri and r.twot and r.ip2 and r.join and
  ↪  r.borel=entry.borel_order) then Error("independent recognition failed");fi;
 if Group(Concatenation(List(entry.panels,GeneratorsOfGroup)))<>G then Error("panel
  ↪  generation failed");fi;
 for j in [1..Length(entry.panels)] do
  if Intersection(entry.parabolics{Difference([1..Length(entry.panels)],[j])})<>entry.panel
   ↪  s[j] then Error("panel reconstruction failed");fi;
 od;
 Print("WITNESS id=",i," rank=",Length(entry.panels)," borel=",r.borel,"
  ↪  parabolic_orders=",r.orders," panel_orders=",r.panels," all_axioms=true\n");
od;
Print("ALL_WITNESSES_PASS count=",Length(J2Representatives),"\n");
QUIT;
```

8 The complete common geometry predicates

```
# Standalone original arbitrary-rank checks; no supplementary archive required.
IsMaximalPair:=function(K,H)
 local d,A,m;
 d:=Index(K,H);
 if d=1 then return false;fi;
 if IsPrimeInt(d) then return true;fi;
 if Size(K)>1000 then
  return ForAny(ConjugacyClassesMaximalSubgroups(K),m->Size(Representative(m))=Size(H) and
   ↪  RepresentativeAction(K,H,Representative(m),OnPoints)<>fail);
 fi;
 A:=Action(K,RightCosets(K,H),OnRight);
 return IsPrimitive(A,[1..d]);
end;
VerifyGeometry:=function(G,pars)
 local I,ss,sub,J,K,remaining,rr,i,j,panel,firm,rc,rwpri,twot,ip2,ft,rows,common,expected,T
  ↪  ,A,B,C,x,left,right,digon,dis,joins,pans,joinok,failures;
 I:=[1..Length(pars)];ss:=Combinations(I);
 if not ForAll(pars,H->IsSubgroup(G,H) and H<>G) then Error("input subgroup membership");fi;
 sub:=List(ss,J->Intersection(Concatenation([G],pars{J})));
 firm:=true;rc:=true;rwpri:=true;twot:=true;ip2:=true;ft:=true;joinok:=true;failures:=[];
 for rr in [1..Length(ss)] do
  J:=ss[rr];K:=sub[rr];remaining:=Difference(I,J);
  if Length(remaining)>0 then
   if not ForAny(remaining,i->IsMaximalPair(K,Intersection(K,pars[i]))) then
    ↪  rwpri:=false;fi;
  fi;
  if Length(remaining)>=2 then
   joins:=Group(Concatenation(List(remaining,i->GeneratorsOfGroup(Intersection(K,pars[i])))
    ↪  ));
   if joins<>K then rc:=false;fi;
  fi;
  if Length(remaining)=1 then
   B:=Intersection(K,pars[remaining[1]]);
   if Index(K,B)<2 then firm:=false;fi;
   if Length(DoubleCosets(K,B,B))<>2 then twot:=false;fi;
  fi;
  if Length(remaining)=2 then
   A:=Intersection(K,pars[remaining[1]]);B:=Intersection(K,pars[remaining[2]]);C:=Intersect
    ↪  ion(A,B);
   digon:=Size(A)*Size(B)=Size(K)*Size(C);
   if not digon then
    left:=Set(List(RightTransversal(A,C),h->RightCoset(B,h)));
    for dis in DoubleCosets(B,C,C) do
```

```
     x:=Representative(dis);
     if not x in C then
      right:=Set(List(RightTransversal(A,C),h->RightCoset(B,h*x)));
      if Length(Intersection(left,right))<>1 then ip2:=false;fi;
     fi;
    od;
   fi;
  fi;
 od;
 # Typed Helly: every partial base flag has precisely the expected common orbit.
 for i in I do
  remaining:=Difference(I,[i]);rows:=[];
  for j in remaining do
   T:=Intersection(pars[i],pars[j]);
   rows[j]:=Set(List(RightTransversal(pars[j],T),x->RightCoset(pars[i],x)));
  od;
  for J in Combinations(remaining) do
   if Length(J)>=2 then
    common:=Length(Intersection(List(J,j->rows[j])));
    T:=sub[Position(ss,J)];expected:=Index(T,Intersection(T,pars[i]));
    if common<>expected then ft:=false;Add(failures,[i,J,common,expected]);fi;
   fi;
  od;
 od;
 # Panels reconstruct every parabolic by joins in the common torsor.
 B:=Intersection(pars);pans:=List(I,i->Intersection(Concatenation([G],pars{Difference(I,[i]
  ↪ )})));
 for J in ss do
  joins:=ClosureGroup(B,Group(Concatenation([()],Concatenation(List(Difference(I,J),i->Gene
   ↪ ratorsOfGroup(pans[i])))))));
  if joins<>sub[Position(ss,J)] then joinok:=false;fi;
 od;
 return rec(orders:=List(pars,Size),borel:=Size(B),panels:=List(pans,Size),firm:=firm,rc:=r
  ↪ c,rwpri:=rwpri,twot:=twot,ip2:=ip2,ft:=ft,join:=joinok,failures:=failures);
end;
```